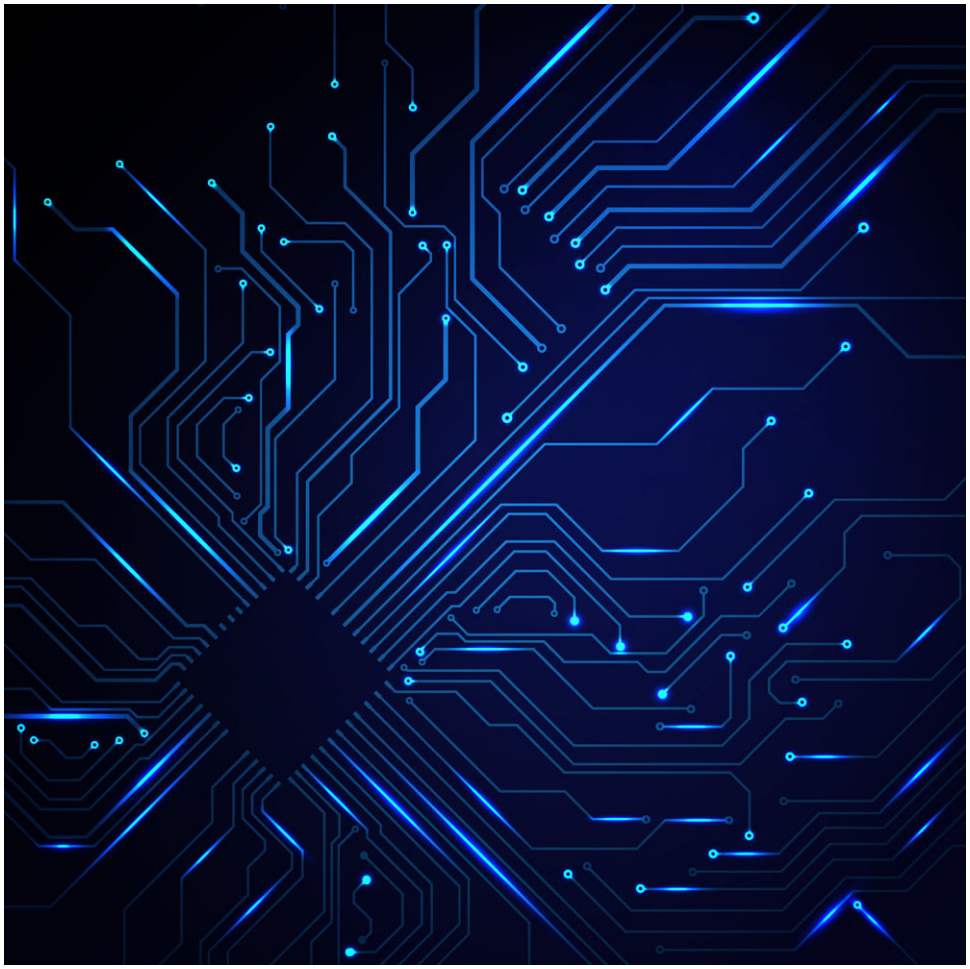


LAB6
COMBINATIONAL LOGIC

Name : 黃翊宏　　Student ID : 01357101

Note

1. All files must be converted to PDF format before submission.
2. Submission deadline: Next Tuesday at 1:00 PM
3. Each person submits one assignment.

※Please use Win + Shift + S for screenshots. Do not take photos of the screen.

Lab6-1 :

Main Code Screenshot

C: > Users > USER > Downloads > tempWeek10 > de0_empty > design > decoder.sv

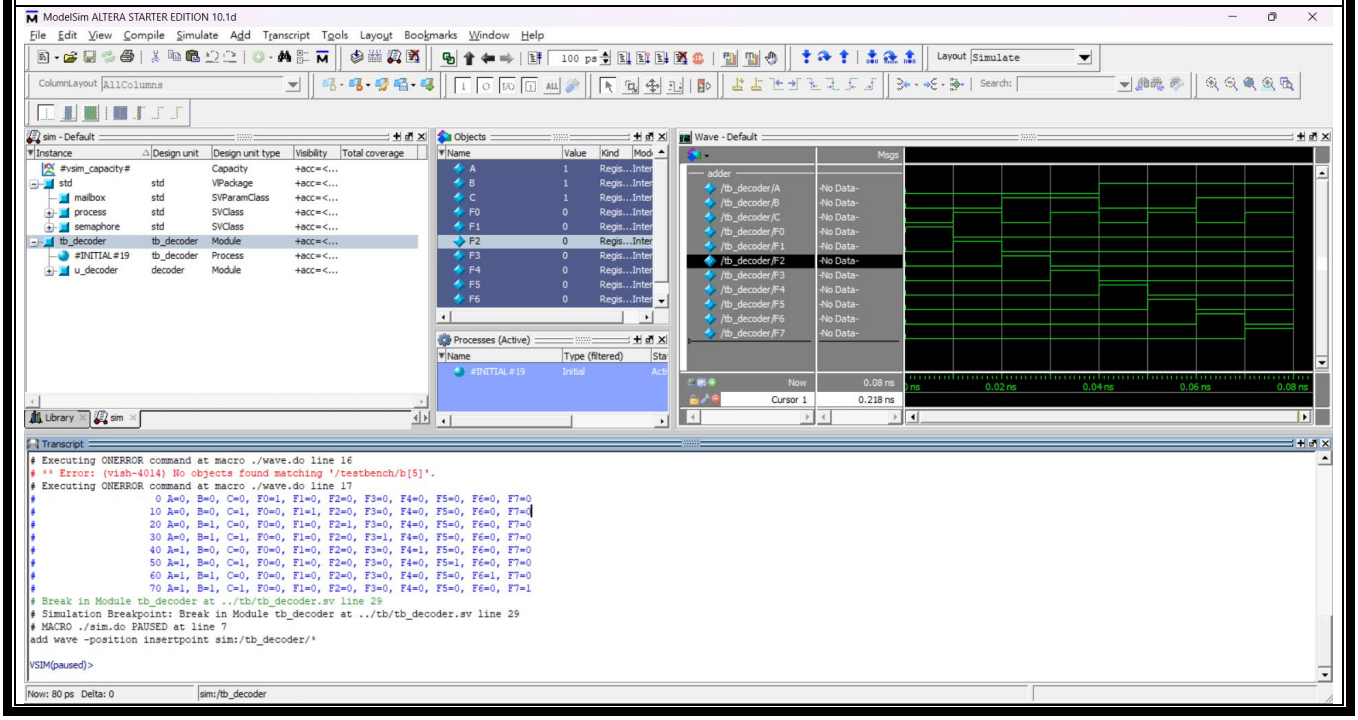
```
1   module decoder(
2       input A,
3       input B,
4       input C,
5       output logic F0,
6       output logic F1,
7       output logic F2,
8       output logic F3,
9       output logic F4,
10      output logic F5,
11      output logic F6,
12      output logic F7
13  );
14
15      logic X0, X1, X2, X3, X4, X5, X6, X7;
16
17      and AND0 (X0, ~A, ~B);
18      and AND1 (F0, X0, ~C);
19      and AND2 (X1, ~A, ~B);
20      and AND3 (F1, X1, C);
21      and AND4 (X2, ~A, B);
22      and AND5 (F2, X2, ~C);
23      and AND6 (X3, ~A, B);
24      and AND7 (F3, X3, C);
25      and AND8 (X4, A, ~B);
26      and AND9 (F4, X4, ~C);
27      and AND10 (X5, A, ~B);
28      and AND11 (F5, X5, C);
29      and AND12 (X6, A, B);
30      and AND13 (F6, X6, ~C);
31      and AND14 (X7, A, B);
32      and AND15 (F7, X7, C);
33
34  endmodule
```

Testbench Screenshot

C: > Users > USER > Downloads > tempWeek10 > de0_empty > simulation > tb > V tb_decoder.sv

```
1  module tb_decoder;
2      logic A, B, C;
3      logic F0, F1, F2, F3, F4, F5, F6, F7;
4
5      decoder u_decoder(
6          .A(A),
7          .B(B),
8          .C(C),
9          .F0(F0),
10         .F1(F1),
11         .F2(F2),
12         .F3(F3),
13         .F4(F4),
14         .F5(F5),
15         .F6(F6),
16         .F7(F7)
17     );
18
19     initial
20     begin
21             A = 0; B = 0; C= 0;
22         #10 A = 0; B = 0; C = 1;
23         #10 A = 0; B = 1; C = 0;
24         #10 A = 0; B = 1; C = 1;
25         #10 A = 1; B = 0; C = 0;
26         #10 A = 1; B = 0; C = 1;
27         #10 A = 1; B = 1; C = 0;
28         #10 A = 1; B = 1; C = 1;
29         #10 $stop;
30     end
31
32
33     initial
34     begin
35         $monitor($time, " A=%b, B=%b, C=%b, F0=%b, F1=%b, F2=%b, F3=%b, F4=%b, F5=%b, F6=%b, F7=%b",A,B,C,F0,F1,F2,F3,F4,F5,F6,F7);
36     end
37
38 endmodule
```

Waveform and Result Screenshot



Lab6-2 :

Main Code Screenshot

C: > Users > USER > Downloads > tempWeek10 > de0_empty > design > multiplexer.sv

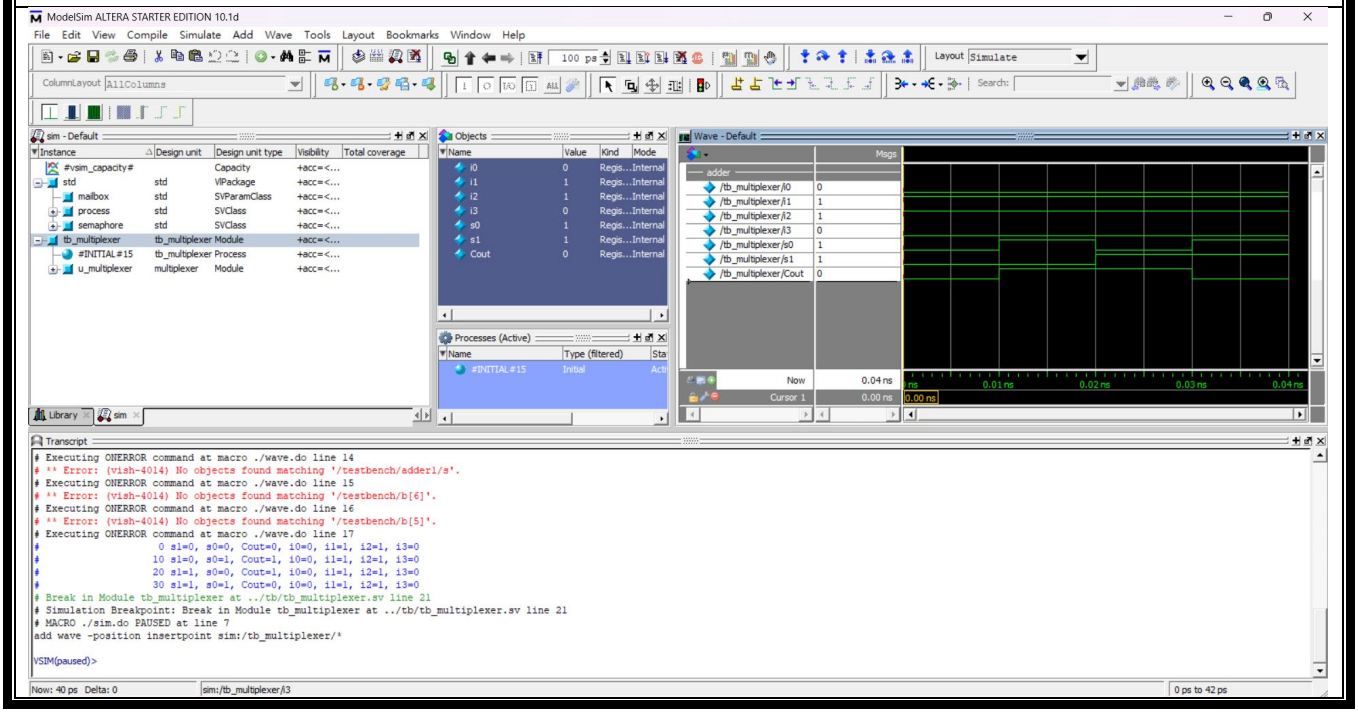
```
module multiplexer(
    input s0,
    input s1,
    input i0,
    input i1,
    input i2,
    input i3,
    output logic Cout
);

    logic X0, X1, X2, X3;
    logic Y0, Y1, Y2, Y3;
    logic Z0, Z1;

    and AND0(X0, i0, ~s1);
    and AND1(Y0, X0, ~s0);
    and AND2(X1, i1, ~s1);
    and AND3(Y1, X1, s0);
    and AND4(X2, i2, s1);
    and AND5(Y2, X2, ~s0);
    and AND6(X3, i3, s1);
    and AND7(Y3, X3, s0);
    or OR0(Z0, Y0, Y1);
    or OR1(Z1, Z0, Y2);
    or OR2(Cout, Z1, Y3);

endmodule
```

Testbench Screenshot

C: > Users > USER > Downloads > tempWeek10 > de0_empty > simulation > tb > V tb_multiplexer.sv

```
1   module tb_multiplexer;
2       logic i0, i1, i2, i3, s0, s1;
3       logic Cout;
4
5       multiplexer u_multiplexer(
6           .i0(i0),
7           .i1(i1),
8           .i2(i2),
9           .i3(i3),
10          .s0(s0),
11          .s1(s1),
12          .Cout(Cout)
13      );
14
15      initial
16      begin
17              s1 = 0; s0 = 0; i0 = 0; i1 = 1; i2 = 1; i3 = 0;
18          #10 s1 = 0; s0 = 1; i0 = 0; i1 = 1; i2 = 1; i3 = 0;
19          #10 s1 = 1; s0 = 0; i0 = 0; i1 = 1; i2 = 1; i3 = 0;
20          #10 s1 = 1; s0 = 1; i0 = 0; i1 = 1; i2 = 1; i3 = 0;
21          #10 $stop;
22      end
23
24
25      initial
26      begin
27          $monitor($time, " s1=%b, s0=%b, Cout=%b, i0=%b, i1=%b, i2=%b, i3=%b",s1,s0,Cout,i0,i1,i2,i3);
28      end
29
30  endmodule
```

Waveform and Result Screenshot



Lab6-3 :

Main Code Screenshot

C: > Users > USER > Downloads > tempWeek10 > de0_empty > design > adder4B.sv

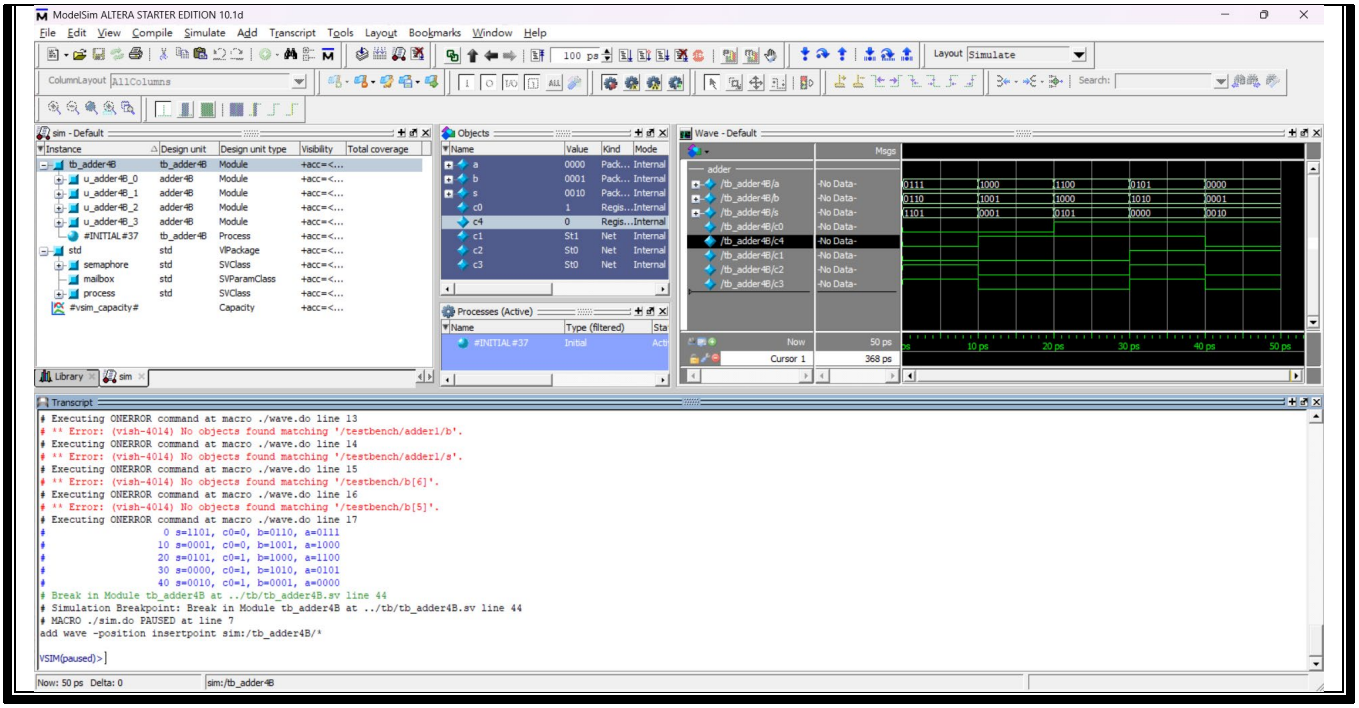
```
1  module adder4B(
2      input [3:0]a,
3      input [3:0]b,
4      input c0,
5      output logic c4,
6      output logic s
7  );
8
9      logic c1, c2, c3;
10
11     xor xor1(c1,a,b);
12     xor xor2(s,c1,c0);
13     and and1(c2,c1,c0);
14     and and2(c3,a,b);
15     or or1(c4,c2,c3);
16
17 endmodule
```

Testbench Screenshot

C: > Users > USER > Downloads > tempWeek10 > de0_empty > simulation > tb > V tb_adder4B.sv

```
module tb_adder4B;
    logic [3:0]a,b,s;
    logic c0, c4;

    adder4B u_adder4B_0(
        .a(a[0]),
        .b(b[0]),
        .c0(c0),
        .c4(c1),
        .s(s[0])
    );

    adder4B u_adder4B_1(
        .a(a[1]),
        .b(b[1]),
        .c0(c1),
        .c4(c2),
        .s(s[1])
    );

    adder4B u_adder4B_2(
        .a(a[2]),
        .b(b[2]),
        .c0(c2),
        .c4(c3),
        .s(s[2])
    );

    adder4B u_adder4B_3(
        .a(a[3]),
        .b(b[3]),
        .c0(c3),
        .c4(c4),
        .s(s[3])
    );

    initial
    begin
            c0 = 0; a[0] = 1; a[1] = 1; a[2] = 1; a[3] = 0; b[0] = 0; b[1] = 1; b[2] = 1; b[3] = 0;
        #10 c0 = 0; a[0] = 0; a[1] = 0; a[2] = 0; a[3] = 1; b[0] = 1; b[1] = 0; b[2] = 0; b[3] = 1;
        #10 c0 = 1; a[0] = 0; a[1] = 0; a[2] = 1; a[3] = 1; b[0] = 0; b[1] = 0; b[2] = 0; b[3] = 1;
        #10 c0 = 1; a[0] = 1; a[1] = 0; a[2] = 1; a[3] = 0; b[0] = 0; b[1] = 1; b[2] = 0; b[3] = 1;
        #10 c0 = 1  ; a[0] = 0; a[1] = 0; a[2] = 0; a[3] = 0; b[0] = 1; b[1] = 0; b[2] = 0; b[3] = 0;
        #10 $stop;
    end


    initial
    begin
        $monitor($time, " s=%b, c0=%b, b=%b, a=%b",s,c0,b,a);
    end

endmodule
```

Waveform and Result Screenshot


ModelSim ALTERA STARTER EDITION 10.1d
File Edit View Compile Simulate Add Transcript Tools Layout Bookmarks Window Help
sim - Default
Objects
Wave - Default
Processes (Active)
Transcript
Executing ONERROR command at macro ./wave.do line 13
** Error: (vish-4014) No objects found matching '/testbench/adder1/b'.
Executing ONERROR command at macro ./wave.do line 14
** Error: (vish-4014) No objects found matching '/testbench/adder1/s'.
Executing ONERROR command at macro ./wave.do line 15
** Error: (vish-4014) No objects found matching '/testbench/b[6]'.
Executing ONERROR command at macro ./wave.do line 16
** Error: (vish-4014) No objects found matching '/testbench/b[5]'.
Executing ONERROR command at macro ./wave.do line 17
0 s=1101, c0=0, b=0110, a=0111
10 s=0001, c0=0, b=1001, a=1000
20 s=0101, c0=1, b=1000, a=1100
30 s=0000, c0=1, b=1010, a=0101
40 s=0010, c0=1, b=0001, a=0000
Break in Module tb_adder4B at ../tb/tb_adder4B.sv line 44
Simulation Breakpoint: Break in Module tb_adder4B at ../tb/tb_adder4B.sv line 44
MACRO ./sim.do PAUSED at line 7
add wave -position insertpoint sim:/tb_adder4B/*
VSIM(paused)>
Now: 50 ps Delta: 0
sim:/tb_adder4B

Lab Summary and Reflection:

Summary

In this week’ s digital logic laboratory, we employed Quartus to author and synthesize a series of combinational circuits using System Verilog. We began by defining each module—namely a 3-to-8 decoder, a 4-to-1 multiplexer, and a 4-bit ripple-carry adder—specifying clear input/output ports and leveraging the logic data type for internal signals □cite□turn0file1□turn0file2□. For each design, we wrote gate-level descriptions that mirrored the textbook symbols, then constructed comprehensive test benches to exhaustively exercise all possible input combinations. Through ModelSim simulation driven by .do scripts, we generated waveforms to verify that the decoder correctly asserted one of eight outputs based on three inputs, the multiplexer faithfully routed the selected data line to its output, and the adder produced accurate sum and carry bits across four-bit operands □cite□turn0file2□.

Reflection

Working on these System Verilog implementations deepened my appreciation for HDL design flow and reinforced best practices in module naming, signal declaration, and operator precedence—insights underscored by the variety of System Verilog operators covered during the exercise □cite□turn0file3□. Debugging mismatches between expected and simulated results taught me to pay close attention to sensitivity lists in always blocks and to verify bit widths at every stage. Moreover, integrating Quartus for synthesis and ModelSim for simulation bolstered my confidence in FPGA toolchains and prepared me to tackle more complex sequential systems. Moving forward, I plan to extend these skills toward a 4-bit adder/subtractor with overflow detection and to explore resource optimization techniques for improved logic utilization.